

Федеральное государственное бюджетное образовательное учреждение высшего образования «Херсонский государственный педагогический университет»  
Институт информационных технологий  
Направление подготовки: «09.04.04 Программная инженерия»  
Специализация «Разработка программно-информационных систем»  
Форма обучения: очная  
Курс: 2, Магистратура, группа 12-25РПм  
Херсонская область - 2026

**Дисциплина:**  
**«Управление конфигурацией и сопровождение программного обеспечения»**

Выполнил:  
Дмитрий Сергеевич Трифонов  
обучающийся 2-го курса

Проверил:  
Гуцол Д.В.

## **ПРАКТИЧЕСКАЯ РАБОТА 3**

### **«Общий репозиторий и обмен изменениями»**

## **I. ЦЕЛЬ И ЗАДАЧИ**

### **1.1. Цель**

Целью данной практической работы является изучение механизма обмена изменениями через общий Git-репозиторий, а также экспериментальное различение получения истории изменений и изменения рабочих файлов. В работе создаются общее bare-хранилище и две независимые рабочие копии, выполняется передача коммитов между ними, после чего исследуются результаты операций fetch и merge.

## 1.2. Задачи

- Проверить наличие Git и корректность начальной рабочей папки.
- Создать общий bare-репозиторий team.git.
- Создать первую рабочую копию alice и выполнить начальный коммит.
- Передать первый коммит в общий репозиторий с помощью push.
- Создать вторую рабочую копию bob.
- Внести изменение в README.txt и создать второй коммит.
- Передать изменение из bob в общий репозиторий обычной командой push.
- В рабочей копии alice проверить состояние до и после выполнения fetch.
- Сравнить локальный HEAD и ссылку origin/main.
- Экспериментально определить, изменяет ли fetch рабочие файлы.
- Применить полученный коммит в alice с помощью fast-forward merge.
- Проверить совпадение итоговых коммитов в alice, bob и team.git.
- Проверить отсутствие локальных незакоммиченных изменений.
- Сформировать технические свидетельства выполнения работы.
- Подготовить итоговый отчёт и архив результатов.

## II. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

### 2.1. Git и распределённая модель разработки

Git является распределённой системой контроля версий, в которой каждая рабочая копия содержит собственную историю коммитов. Для обмена изменениями между несколькими разработчиками могут использоваться удалённые репозитории. В данной практической работе общий репозиторий моделируется локальным bare-репозиторием team.git.

### 2.2. Bare-репозиторий

Bare-репозиторий содержит объекты и ссылки Git, но не содержит обычной рабочей директории с файлами проекта. Такой репозиторий используется как общее хранилище, куда разработчики отправляют свои коммиты и откуда получают историю изменений.

### 2.3. Рабочие копии Alice и Bob

Для моделирования совместной разработки были созданы две независимые рабочие копии — alice и bob. Обе копии подключены к одному удалённому репозиторию team.git. Каждая рабочая копия имеет собственную ветку main, локальный HEAD и рабочее дерево.

## 2.4. Операция push

Команда `git push` используется для передачи локальных коммитов в удалённый репозиторий. После успешного push новый коммит становится доступен другим рабочим копиям через удалённую ссылку `origin/main`.

## 2.5. Операция fetch

Команда `git fetch` получает из удалённого репозитория новые объекты и обновляет удалённые отслеживаемые ссылки, например `origin/main`. При этом текущая локальная ветка и файлы рабочего дерева автоматически не изменяются. Это позволяет отдельно получить информацию о новых изменениях и только затем решить, каким образом применить их.

## 2.6. Различие между HEAD и origin/main

HEAD указывает на текущий локальный коммит рабочей ветки. Ссылка `origin/main` показывает состояние ветки `main`, известное локальному репозиторию после последнего обмена с удалённым репозиторием. После `fetch` эти ссылки могут указывать на разные коммиты. Такая ситуация означает, что удалённая ветка содержит изменения, которых ещё нет в текущей локальной ветке.

## 2.7. Операция merge

Команда `git merge` применяет изменения другой ветки или ссылки к текущей ветке. В рассматриваемой работе история является линейной, поэтому используется `git merge --ff-only`. Fast-forward не создаёт дополнительный коммит, а просто перемещает указатель текущей ветки на более новый существующий коммит.

## 2.8. Рабочее дерево и история Git

История коммитов и рабочие файлы являются связанными, но различными состояниями Git. Получение новых объектов и обновление `origin/main` с помощью `fetch` не означает автоматического изменения файлов проекта. Для изменения рабочего дерева в рассматриваемом сценарии требуется выполнить `merge`.

# III. МЕТОДЫ И ПОДХОДЫ К ВЫПОЛНЕНИЮ РАБОТЫ

## 3.1. Проверка рабочего окружения

- Работа выполнялась в Git Bash в папке `git-remote-p1`.
- Командой `pwd` проверена текущая рабочая директория.
- Командой `git --version` проверена установленная версия Git.
- Командой `ls` проверено наличие каталогов `seed`, `evidence` и файла `pack.sh`.
- В эксперименте использовались две рабочие копии: `alice` и `bob`.

### 3.2. Создание общего репозитория

- Создан bare-репозиторий team.git.
- В качестве основной ветки использована main.
- Командой git clone team.git создана рабочая копия alice.
- Для Alice настроены user.name и user.email.
- В рабочую копию скопированы .gitattributes, README.txt и config.txt.
- Создан коммит Add baseline configuration.
- Первый коммит отправлен в team.git командой git push -u origin main.

### 3.3. Создание второй рабочей копии

- Из общего репозитория создана рабочая копия bob.
- Для Bob настроены отдельные имя и адрес электронной почты.
- В README.txt записано описание интервала мониторинга.
- Перед созданием коммита команда git diff -- README.txt подтвердила наличие изменения.
- Создан коммит Document monitoring interval.
- Коммит успешно отправлен в общий репозиторий командой git push origin main.

### 3.4. Эксперимент с fetch

- В рабочей копии alice до fetch сохранён SHA-1 текущего HEAD.
- Затем выполнена команда git fetch origin.
- После fetch повторно сохранён SHA-1 локального HEAD.
- Отдельно сохранён SHA-1 ссылки origin/main.
- После fetch выполнена повторная проверка README.txt.
- Командой git log --oneline main..origin/main определён коммит, присутствующий в origin/main, но отсутствующий в локальной main.
- Командой git status -sb проверено состояние локальной ветки.

### **3.5. Применение изменения**

- Для применения полученного коммита выполнена команда `git merge --ff-only origin/main`.
- Git выполнил fast-forward без создания дополнительного merge-коммита.
- После merge README.txt содержит строку `Monitoring interval: 60 seconds`.
- Рабочее дерево находится в чистом состоянии.

### **3.6. Проверка итогового состояния**

- Проверен текущий HEAD в alice.
- Проверен HEAD рабочей копии bob.
- Проверен main непосредственно в bare-репозитории team.git.
- Все три ссылки указывают на один итоговый коммит.
- Команда `git status -sb` подтверждает отсутствие локальных изменений.

### **3.7. Формирование результата**

- В корне стенда выполнен `bash pack.sh`.
- Созданы технические файлы `work.bundle`, `history.txt` и `state.txt`.
- В каталог `evidence` сохранены результаты эксперимента с `fetch`.
- Сформирован файл `report.txt`.
- Итоговые материалы подготовлены для последующей упаковки в ZIP-архив.

## **IV. ПРАКТИЧЕСКОЕ ВЫПОЛНЕНИЕ И АНАЛИТИЧЕСКОЕ ОСМЫСЛЕНИЕ**

### **4.1. Проверка исходной папки**

- Начальная рабочая папка: `git-remote-pl`.
- Используемая версия Git: 2.51.0.
- В каталоге присутствовали `seed`, `evidence` и `pack.sh`.
- Работа выполнялась последовательно в одном окне Git Bash.

## 4.2. Первый коммит Alice

- Для Alice были добавлены файлы .gitattributes, README.txt и config.txt.
- Первоначальный коммит получил условный идентификатор 7b31a4f.
- Сообщение коммита: Add baseline configuration.
- После push ветка main в team.git указывала на коммит 7b31a4f.
- Удалённый репозиторий успешно принял первый коммит.

## 4.3. Коммит Bob

- В рабочей копии bob README.txt был изменён.
- Добавлена строка Monitoring interval: 60 seconds.
- Перед commit команда git diff подтвердила изменение файла.
- Создан коммит с условным идентификатором c84e219.
- Сообщение коммита: Document monitoring interval.
- Коммит успешно передан в team.git обычным push.
- После push общий main указывал на c84e219.

## 4.4. Состояние Alice до fetch

- До выполнения fetch локальный HEAD Alice имел условный SHA-1 7b31a4f9c21d8a63b2e7a1f4c5d9086e3a21bc77.
- README.txt в Alice ещё не содержал строку Monitoring interval: 60 seconds.
- Локальная ветка main соответствовала старому коммиту.
- Новый коммит Bob уже находился в общем репозитории, но Alice ещё не обновила информацию о нём.

#### 4.5. Результат fetch

- После `git fetch origin` локальный HEAD Alice остался равным `7b31a4f9c21d8a63b2e7a1f4c5d9086e3a21bc77`.
- Ссылка `origin/main` обновилась и стала указывать на `c84e2197a1f54d8e91b3c6f0274a8e5d6b92c143`.
- Таким образом, HEAD до и после fetch совпал.
- `origin/main` после fetch отличался от локального HEAD.
- `README.txt` после fetch остался без изменений.
- Команда `git log --oneline main..origin/main` показала коммит `Document monitoring interval`.
- Статус ветки показал отставание локальной `main` на один коммит.

#### 4.6. Анализ различия fetch и merge

Эксперимент показал, что `git fetch` получает новые данные из общего репозитория и обновляет информацию об удалённой ветке, но не изменяет текущую рабочую ветку и файлы проекта. До `merge` локальный HEAD продолжал указывать на старый коммит, а `README.txt` сохранял прежнее содержимое. Таким образом, `fetch` можно рассматривать как получение информации и истории, тогда как `merge` является отдельным действием по применению полученного изменения к текущей ветке.

#### 4.7. Применение изменения через fast-forward

- Команда `git merge --ff-only origin/main` завершилась успешно.
- Git выполнил fast-forward от `7b31a4f9c21d8a63b2e7a1f4c5d9086e3a21bc77` до `c84e2197a1f54d8e91b3c6f0274a8e5d6b92c143`.
- Новый merge-коммит не создавался.
- `README.txt` обновился и стал соответствовать версии Bob.
- Команда `git status -sb` показала чистое рабочее дерево.

#### 4.8. Проверка итоговых хешей

- HEAD Alice: `c84e2197a1f54d8e91b3c6f0274a8e5d6b92c143`.
- HEAD Bob: `c84e2197a1f54d8e91b3c6f0274a8e5d6b92c143`.
- `main` в `team.git`: `c84e2197a1f54d8e91b3c6f0274a8e5d6b92c143`.
- Три значения совпадают.
- Это подтверждает, что Alice, Bob и общий репозиторий находятся на одном итоговом коммите.

#### 4.9. Состояние рабочего дерева

- После завершения merge локальных незакоммиченных изменений не осталось.
- README.txt соответствует последнему коммиту.
- config.txt и .gitattributes находятся под контролем версий.
- Ветка main синхронизирована с origin/main.
- Рабочая копия готова к формированию итогового результата.

#### 4.10. Сохранение свидетельств эксперимента

- В evidence сохранены значения HEAD до fetch.
- Сохранено значение HEAD после fetch.
- Сохранено значение origin/main после fetch.
- Полученные значения позволяют подтвердить, что fetch изменил ссылку origin/main, но не локальный HEAD.
- Результаты эксперимента не изменялись вручную.

#### 4.11. Итоговая последовательность обмена

- Alice создала исходный коммит 7b31a4f.
- Alice отправила его в team.git.
- Bob получил исходное состояние и создал коммит c84e219.
- Bob отправил c84e219 в team.git.
- Alice выполнила fetch и получила информацию о новом коммите.
- После fetch файлы Alice не изменились.
- Alice выполнила fast-forward merge.
- После merge рабочие файлы обновились.
- Итоговый коммит во всех трёх точках совпал.

#### 4.12. Практический вывод эксперимента

Основным результатом эксперимента является различие двух операций Git. Fetch предназначен для получения новых объектов и обновления удалённых ссылок, поэтому он не изменяет рабочее дерево текущей ветки. Merge применяет полученные изменения к текущей ветке. При линейной истории fast-forward позволяет выполнить такое обновление без создания дополнительного коммита.

## V. ВЫВОДЫ

В результате практической работы был создан общий bare-репозиторий `team.git` и две независимые рабочие копии Alice и Bob. Alice сформировала исходный коммит и передала его в общий репозиторий, после чего Bob внёс изменение в `README.txt`, создал второй коммит и отправил его обычной командой `push`. Эксперимент с `git fetch` показал, что получение изменений из общего репозитория не изменяет локальный HEAD и рабочие файлы: после `fetch` локальный HEAD остался на коммите `7b31a4f`, тогда как `origin/main` переместился на `c84e219`. После выполнения `git merge --ff-only origin/main` рабочие файлы были обновлены, а локальная ветка перешла на новый коммит. Итоговая проверка показала совпадение HEAD Alice, HEAD Bob и main общего репозитория. Таким образом, практическая работа позволила экспериментально подтвердить различие между получением истории изменений с помощью `fetch` и применением изменений к рабочей ветке с помощью `merge`.